



Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Student Name	Om Mule
PRN No	202401040325
Roll No	147
Program	Computer Engineering
Year	Third Year
Division & Batch	B – B3
Subject	Operating Systems Laboratory (2304311L)
Assignment No	Two

Problem Statement:

Develop a program to demonstrate process creation, termination, and communication using PIPE as an IPC (Interprocess Communication) mechanism.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid;
    char write_msg[] = "Data sent through the IPC pipe!";
    char read_msg[100];

    if (pipe(fd) == -1) {
        perror("Pipe creation failed");
        return EXIT_FAILURE;
    }

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return EXIT_FAILURE;
    }

    if (pid > 0) {
        close(fd[0]);

        printf("[Parent] Sending message...\n");
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]);

        wait(NULL);
        printf("[Parent] Child has terminated. Parent exiting.\n");
    } else {
        close(fd[1]);

        read(fd[0], read_msg, sizeof(read_msg));
        printf("[Child] Received message: '%s'\n", read_msg);
        close(fd[0]);
    }
}
```

```

    } else {
        close(fd[1]);

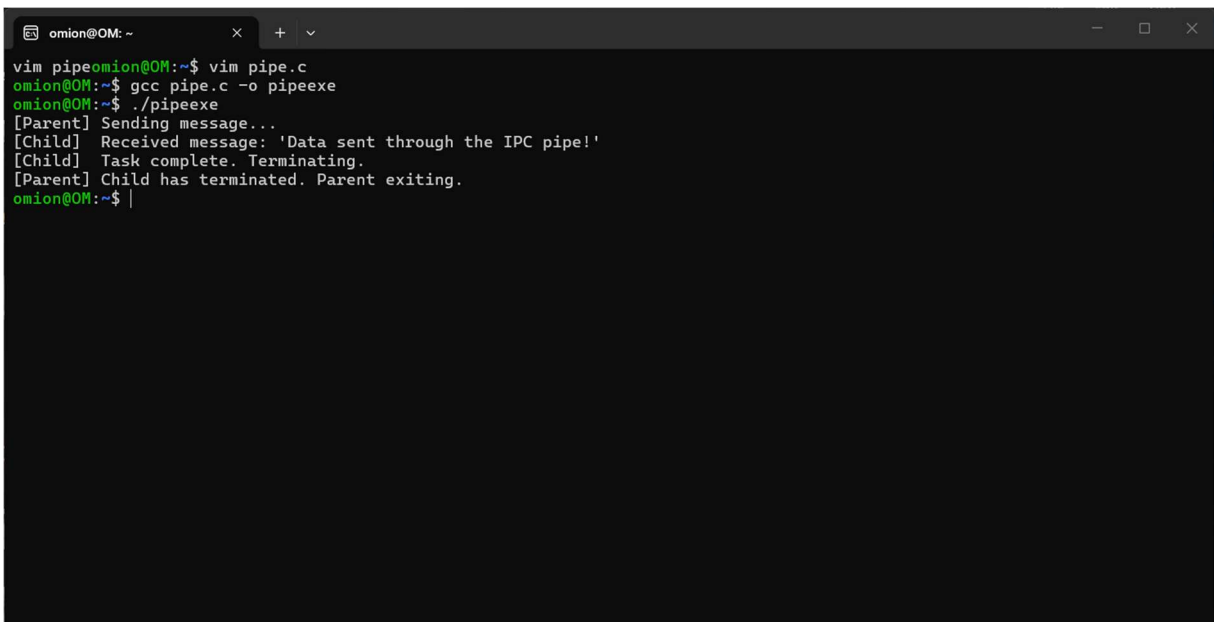
        read(fd[0], read_msg, sizeof(read_msg));
        printf("[Child] Received message: '%s'\n", read_msg);
        close(fd[0]);

        printf("[Child] Task complete. Terminating.\n");
        exit(EXIT_SUCCESS);
    }

    return EXIT_SUCCESS;
}

```

Output of the program:



```

omion@OM: ~
vim pipeomion@OM:~$ vim pipe.c
omion@OM:~$ gcc pipe.c -o pipeexe
omion@OM:~$ ./pipeexe
[Parent] Sending message...
[Child] Received message: 'Data sent through the IPC pipe!'
[Child] Task complete. Terminating.
[Parent] Child has terminated. Parent exiting.
omion@OM:~$

```